

Project4 Mario-RL实验报告

本次实验报告分为四个部分：

- 代码编写（在Agent.py和Main.py里编写了代码）
- DQN的TD target如何计算
- 具体实验，训练过程
- 长周期训练和收敛分析

一.代码编写

Agent.py:

- 1.在本文件中我补全了要求的三个函数，从而使得整体训练可以执行，主要是在提示下把每个函数的代码补全
 - 2.在执行1000轮训练的时候，爆显存不够，发现是经验池全在GPU上，占用了太多显存，于是把经验池放到了CPU上，只在Recall的时候放到GPU上
 - 3.在长周期训练的时候发现训练速度过慢，一晚上只训了2000多轮，于是让ai修改了部分代码，比如经验池用List来存储，加快保存checkpoint频率，每次保存后清空经验池，重新积累步数后学习等，关于相应分析具体在实验部分
- 最后上交的是测用了修改后策略的版本，但其实修改后的策略仍存在很大问题，但因为不再继续跑更长周期的训练，没有修改代码。但同时，先前做了一个小规模1000轮训练（把各种参数微调使得最后epision降到1点多），并进行了参照分析，具体见实验部分

Main.py:

为了保证可复现性，便于受控分析，加了随机种子设定的代码

二.DQN的TD target如何计算

$Q(s,a) = \text{reward} + (1 - \text{done.float()}) * \gamma * Q(s')$

$Q(s')$ 是target 网络算的 next_state 最高分

reward是环境反馈的从s采取动作a得到的奖励

三.实验部分

本次作业我总共总了5个实验

总体来说

1.写完代码后直接跑了100轮,第一次跑记录点太少效果似乎很理想，然后第二次跑设置为每轮打印日志，发现更多细节，比如明显波动，通过分析图像，进一步了解了各个指标的含义和DQN算法的思想，同时为了可控分析，这次实验后我设定了随机种子。

2.在设定随机种子后，再跑了100轮作为新的基准。实验得到的结论与第一次基本相同

3.在长周期训练前，先做了一个小规模模拟训练，跑1000轮，微调参数，比如把save_every调低，exploration-decay调低使得1000轮左右epision就降得比较低，对标10000后期情况

长周期训练

这里主要分为两个阶段，对应两个实验

4.初始设定为在3的基础上回调decay,save_every等，初始目标是一晚上跑完做实验分析，但发现一晚上只跑了2000多轮，且越训越慢，发现是经验池越来越大，每次选出数据花费时间过多，同时我在理论上产生疑惑，经验池中那么多低质数据（指的是很早之前产生的数据）是不是不利于自我增强，于是我中断了实验，调整了代码，在第一个checkpoint重新开始训练

5.重新开始训练，训了4000+轮，加上原来的2000+轮，训了7000+轮，发现震荡严重，经过分析认为原来的有些改动不太合理，比如每次记录都清空经验池，模型学的不稳，而且在理论上，我认为其实原来存在低质数据这种说法不太准确，因为在DQN算法下，TD-target是由reward和不断自我增强的冻结model产生的，大量的原始数据保留了环境对于不同情形下返回的reward信息，而DQN就是根据这些reward信息不断自我迭代。由于波动太大，加上发现原来的认知有问题，提前结束了训练，总共跑了7000+轮。

实验具体细节和长周期训练与收敛分析放在

《DQN_Mario_实验报告.docx》里(现已拼接到下面)，该文件的生成为我将 手写实验过程投喂给kimi Agent生成，我对里面的所有内容 进行了核查，确保为我想表达的意思，同时，我会提交手 写实验报告pdf作为参照（包含每个实验具体过程以及一些 理论想法），但看《DQN_Mario_实验报告.docx》已经完 成足够

马里奥场景 DQN 实验报告

深度强化学习课程实践作业

基于 OpenAI Gym 的超级马里奥兄弟强化学习实验

2025 年 5 月

目 录

- 一、具体实验与训练过程.....3**
 - 1.1 实验一：短周期探索性训练（100 episodes）3
 - 1.2 实验二：种子固定下的基准测试（100 episodes）4
 - 1.3 实验三：中等周期参数调优训练（1000 episodes）4
 - 1.4 实验四：全量级长周期训练（10000+ episodes）5
- 二、长周期训练与收敛分析.....6**
 - 2.1 训练指标的整体趋势.....6
 - 2.2 阶段性现象分析.....7
 - 2.3 小规模与长周期训练对比.....8
 - 2.4 实验反思与优化方向.....8

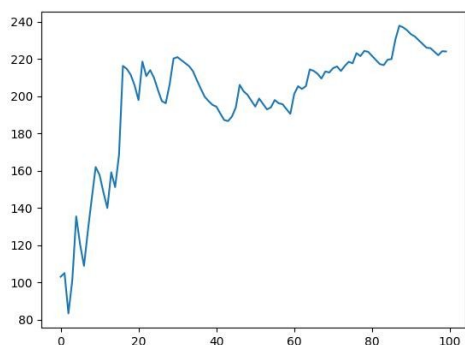
一、具体实验与训练过程

本实验旨在通过深度强化学习中的 DQN (Deep Q-Network) 算法，训练智能体在 OpenAI Gym 环境下的超级马里奥兄弟游戏中完成关卡。实验在 agent.py 中实现了 DQN 所需的三个核心函数：td_estimate（计算当前 Q 值估计）、td_target（计算 TD 目标值）以及 update_Q_online（更新 online 网络参数）。为了全面评估训练效果，本实验设计了四个阶段的实验，从短周期探索性训练逐步过渡到全量级长周期训练。

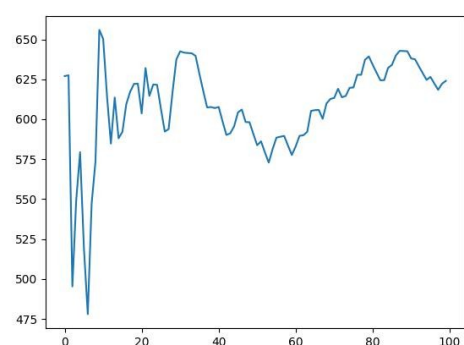
1.1 实验一（分为两次，第二次打印日志更多）：短周期探索性训练（100 episodes）

第一组实验设置了 100 个 episode 的短周期训练，主要用于验证 DQN 代码的基本正确性和环境配置的可用性。在训练过程中，探索率 epsilon 处于较高水平，智能体主要以随机探索的方式进行交互，尚未形成有效的游戏策略，。

观察到的现象包括：Mean Length（每轮存活步数）和 Mean Reward（每轮平均奖励）波动剧烈，这是由于训练数据量不足导致的统计不稳定性。一个值得关注的现象是，在前 50 轮尚未充分训练的阶段，Mean Length 却呈现出上升趋势。经分析，这主要是因为初始阶段经验池（Replay Buffer）中的样本量过少，且早期样本质量较差，极端数据对均值产生了显著影响。随着更多样本的加入，极端数据的影响逐渐被稀释，指标趋于在均值附近波动。此外，Q 值整体呈现平滑的上升趋势，这是因为 Q 值和 loss 经过了两次平均处理（一次是在 episode 内所有学习步的平均，另一次是最近 100 个 episode 的再平均），使得曲线相对平滑。



Length-plot



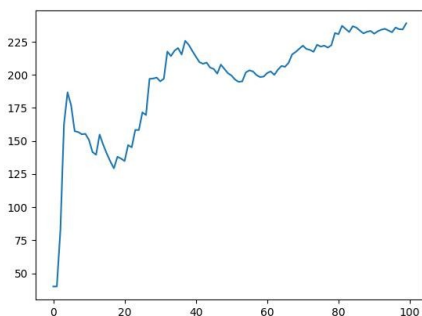
reward-plot

波动很严重，且刚开始时受极端数据影响大

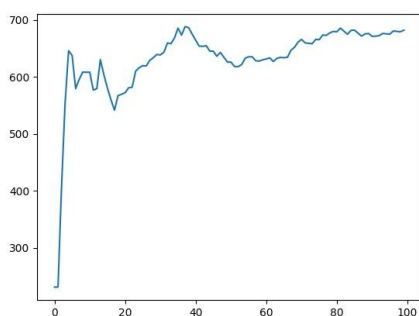
1.2 实验三：种子固定下的基准测试（100 episodes）

为了保证实验的可复现性，在实验三中引入了固定随机种子的设置，以此建立基准对照。实验结果显示，固定种子后的训练曲线与实验二基本一致，验证了训练过程的不确定性。此阶段同样观察到，尽管训练尚未充分展开，Mean Reward 和 Mean Length 仍呈现上升趋势。这进一步印证了之前的分析：初期经验池样本不足且质量偏低，导致初始均值被人为压低；随着更多样本的加入，指标逐渐回归真实均值水平。

此阶段 GPU 占用率仅约 1%~7%，资源利用率较低。分析原因包括：batch size 较小、经验池采样频率不高，以及 learn_every 参数设置导致学习间隔较长。这提示在后续实验中可以考虑调整相关参数以提升训练效率。



基准 100 轮下：length-plot



reward-plot

和实验一结论基本一直

Q 和 loss 上升平滑，分析在实验一

1.3 实验四：中等周期参数调优训练（1000 episodes）

第四组实验将 episode 数提升至 1000，目的是在较完整周期内观察训练趋势，同时对多个超参数进行了适配性调整：

参数	调整前	调整后
save_every	500000	50000
exploration_rate_decay	0.99999975	0.999885
目标	-	1000 轮内 epsilon 衰减至 0.1

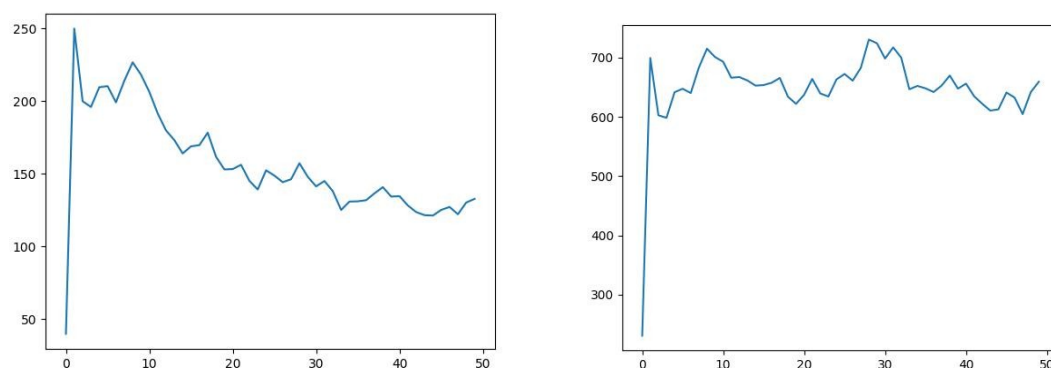
调整后，探索率 epsilon 的目标是在 1000 轮内衰减至 0.1。此阶段 GPU 占用率提升至 10%~11%。训练过程中遇到显存溢出问题（在 300+ 轮时崩溃），通过将回放池数据转移至 CPU 内存的方式解决。重新训练后成功跑完全程。

训练结果显示：Loss 和 Q 值均呈上升趋势。Reward 在度过了初期少数据、极端数据影响较大的阶段后，在中心线附近波动。值得关注的是，Length 呈现下降趋势——这意味着智能体以更少的步数获得了更高的 Reward，表明策略正在向更高效的方向演化，这是一个积极的信号。

1.4 实验四（全量级）：长周期训练（2000+ episodes）

原定目标为完成 10000 个 episode 的全量级训练，但实际运行中受限于训练速度（约 5 分钟跑 20 轮，一晚上仅完成 2000+ 轮），最终完成了超过 2000 轮的训练。在训练过程中，发现经验池过大导致随机采样耗时严重，且池中积累的大量早期低质量数据不利于学习。

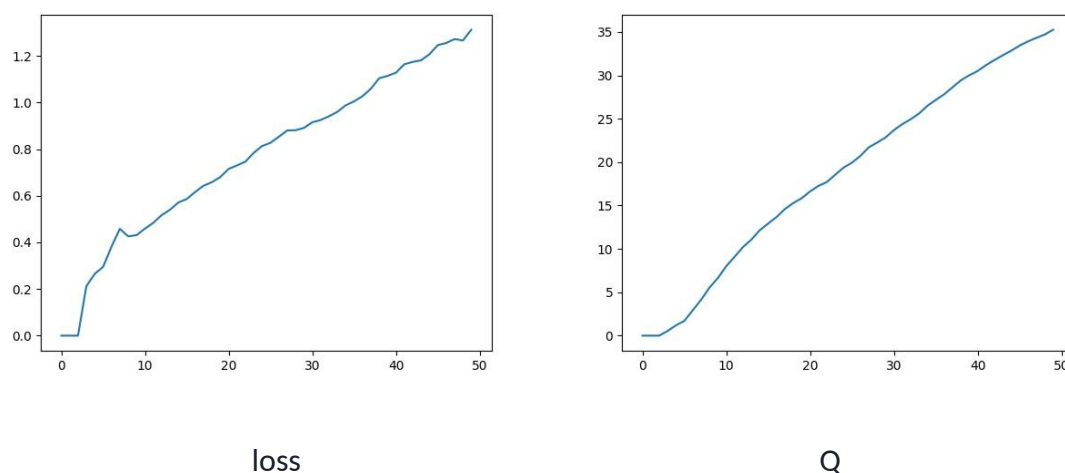
为此，对经验池的存储和读取逻辑进行了重构优化：采用环形缓冲区（List 实现，数组随机访问 $O(1)$ ）替代原有数据结构以提高速度；同时调整了 checkpoint 保存策略（save_every 从 500000 降至 50000），更频繁地保存模型并清空经验池，使更新后的模型能够产生更高质量的数据，形成正反馈循环。



左图是 length，右图是 reward

总体还是比较理想的在 length 降低的情况下，reward 仍然在中心线波动

关于 loss（左图）和 Q（右图）：训练是比较稳定的，对比实验五更明显



loss

Q

二、长周期训练与收敛分析

长周期训练是评估 DQN 算法在复杂游戏环境下性能表现的关键环节。本章节结合 reward、loss、Q value 等核心指标，对训练的收敛性进行系统性分析。

2.1 训练指标的整体趋势

从 reward、Q 值、loss、length 四项指标的综合表现来看，长周期训练整体呈现振荡状态，未观察到明确的收敛迹象。各项指标波动剧烈，说明训练过程尚未达到稳定状态。具体表现如下：

- Reward 波动范围约为 620~770，围绕 680~700 的中心线上下振荡，未形成稳定上升或收敛趋势。
- Q 值在每次 checkpoint 清空经验池后迅速下降，随后在训练过程中缓慢回升，呈现周期性波动。
- Loss 波动同样剧烈，尤其在经验池更新后通常会出现大幅下降，反映模型在新数据上的适应过程。
- Length 与 Reward 呈现类似的波动特征，但总体呈现下降态势，说明智能体趋向于以更短的步数完成游戏。

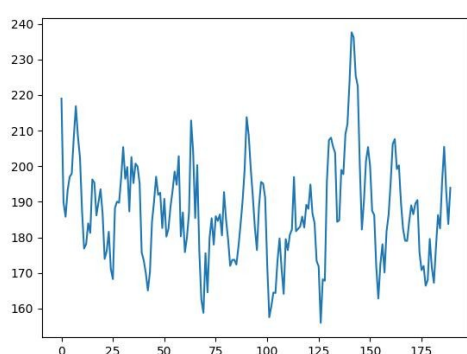
2.2 阶段性现象分析

根据训练日志和指标曲线的观察，可以将长周期训练划分为以下阶段：

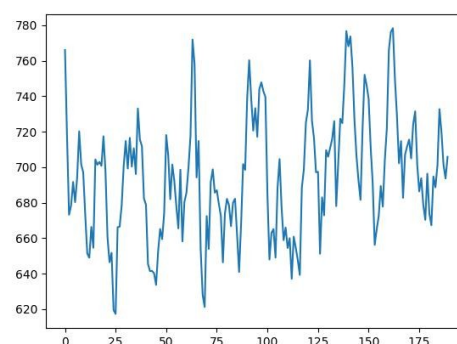
第一阶段（前 3000 轮，实际覆盖全部训练过程）：训练相对平稳，Q 值和 Loss 值均呈现较为稳定的上升趋势。Reward 和 Loss 在各自的中心线附近振荡。然而深入分析发现，此阶段 epsilon 仍然较大（训练至 7000 轮左右仍有约 0.7），智能体主要处于随机探索状态，尚未真正学到有效的游戏策略。指标的波动在均值附近徘徊，实质上反映的是探索行为的随机性，而非策略的实质性进步。

第二阶段（经验池规模扩大后）：当经验池积累到较大规模时，训练速度显著下降。每次从大规模池中随机采样消耗大量时间，且池中保留了较多早期低质量数据，对新知识的学习产生了干扰。这也是促使重构经验池逻辑的直接原因。（实验后我认为低质数据这种说法并不正确，在 DQN 算法下，数据都是没问题的）

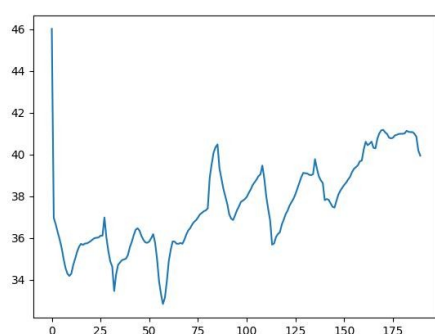
（修改代码后后 4000 轮图像）：



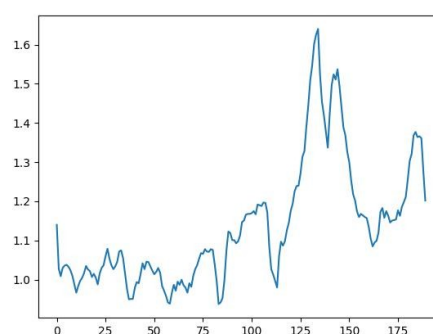
Length



Reward



Q



loss

由于频繁清空经验池，加上随机探索程度太高，几乎没有收敛，几乎是训崩了

但是同时也发现 Mean-reward 波动的中心线比 1000 轮高了很多，通过日志更明显，几乎在 700 左右波动，而 1000 轮小规模训练的时候在 660 左右波动，说明长周期训练还是有点效果的

2.3 小规模与长周期训练对比

将 1000 轮的中等周期训练与 2000+ 轮的长周期训练进行对比分析，可以观察到以下差异：

指标	1000 轮中等周期	2000+ 轮长周期
Reward 波动范围	较小	620 ~ 770
Reward 中心线	约 650	680 ~ 700
收敛迹象	不明显	不明显，但中心线上移
训练稳定性	相对稳定	波动较大

从对比可以看出，长周期训练下 Reward 的波动范围虽然更大，但中心线（680~700）明显高于小规模训练的 650 水平，且通过训练日志可以直观感受到 Reward 的整体提升。这表明长周期训练并非完全没有效果，模型在持续学习中确实积累了一定的经验。

2.4 实验反思与优化方向

基于上述实验观察与分析，总结出以下关键问题和优化方向：

第一，探索率 epsilon 衰减过慢是制约训练效果的核心因素。训练至 7000 轮时 epsilon 仍在 0.7 左右，智能体基本处于随机探索状态，这极大地限制了已学经验的有效利用。后续实验中应加快 epsilon 的衰减速度，使智能体更早进入利用（exploitation）阶段。

第二，经验池的更新频率需要合理控制。过于频繁地清空经验池会导致模型学习不稳定，每次都需要重新适应新数据；但长期不清理又会占用较多空间，对训练效率造成影响。需要在两者之间找到平衡。

反思：其实在 List 储存下，也可以不清洗，因为数据都是没问题的，且更多的数据包含更多的 Reward 信息，使 Mario 在更多场景下的更优策略可以被发掘

第三，save_every 参数的大小直接影响训练稳定性。设置过大会导致自我迭代缓慢；设置过小则会造成"自己追自己"的局面，训练过程不够稳定。

第四，建议在 epsilon 适宜、save_every 合理的配置下，进行更长周期的训练（如完整的 10000+ 轮）。同时保持经验池的相对稳定（减少频繁清洗，或随机清洗部分），让模型有足够的时间从环境反馈中持续学习。虽然这会使单次训练的效率降低，但有助于获得更稳定、更可复现的训练结果。

